```
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression
x=np.array([[1],[2],[3],[4],[5],[6],[7],[8]])
y_reg=np.array([30,40,45,55,65,70,80,90])
y_cls=np.array([0,0,0,0,1,1,1,1])
```

```
lin=LinearRegression()
lin.fit(x,y_reg)
print('Regression output:',lin.predict([[5.5]]))
print('Regression output:',lin.predict([[4.5]]))
print('Regression output:',lin.predict([[9]]))
print('Regression output:',lin.predict([[0.73]]))
```

```
Regression output: [67.76785714]
Regression output: [59.375]
Regression output: [97.14285714]
Regression output: [27.73392857]
```

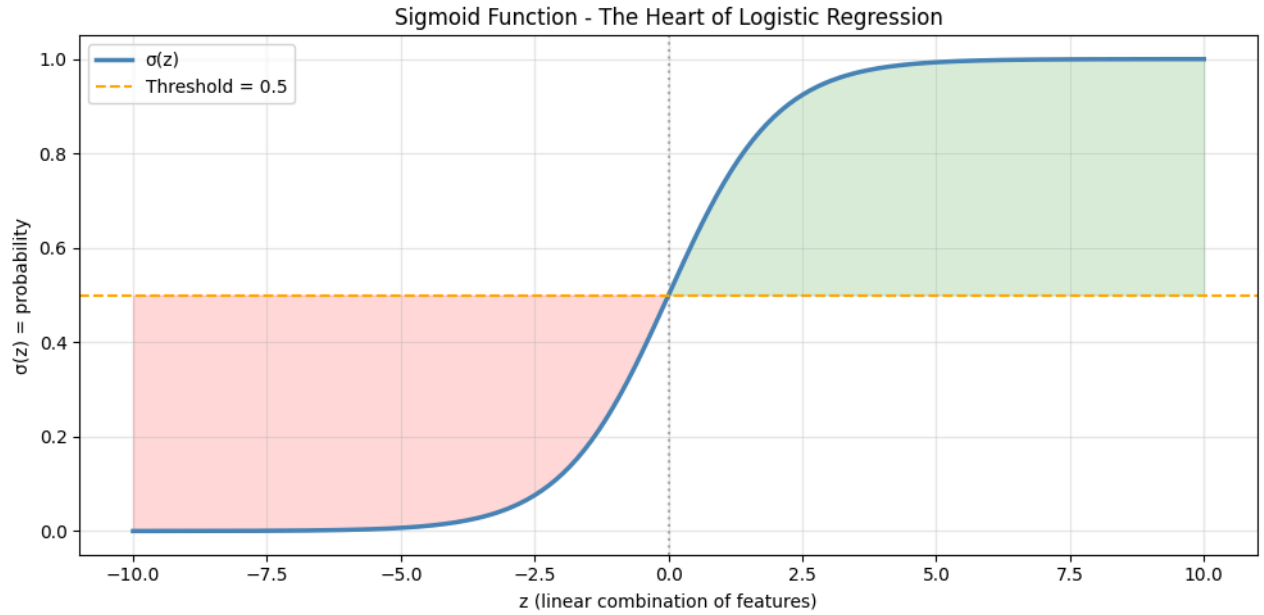
```
log=LogisticRegression()
log.fit(x,y_cls)
print('Regression output:',log.predict([[9]]))
print('Regression output:',log.predict([[5.5]]))
print('Regression output:',log.predict([[4.7]]))
print('Regression output:',log.predict([[3.6]]))
print('Regression output:',log.predict([[0.73]]))
```

```
Regression output: [1]
Regression output: [1]
Regression output: [1]
Regression output: [0]
Regression output: [0]
```

```
print('probability output',log.predict_proba([[9]]))
print('probability output',log.predict_proba([[5.5]]))
print('probability output',log.predict_proba([[4.5]]))
print('probability output',log.predict_proba([[4.7]]))
```

```
probability output [[0.00514748 0.99485252]]
probability output [[0.2368932 0.7631068]]
probability output [[0.50000269 0.49999731]]
probability output [[0.98799263 0.01200737]]
```

```
import numpy as np
import matplotlib.pyplot as plt
def sigmoid(z):
  return 1/(1+np.exp(-z))
z=np.linspace(-10, 10, 300)
prob=sigmoid(z)
plt.figure(figsize=(10,5))
plt.plot(z,prob,color='steelblue',linewidth=2.5,label='σ(z)')
plt.axhline(y=0.5, color='orange', linestyle='--', label='Threshold = 0.5')
plt.axvline(x=0, color='gray', linestyle=':', alpha=0.6)
plt.fill_between(z, prob, 0.5,where=(prob > 0.5),alpha=0.15,color='green')
plt.fill_between(z, prob, 0.5,where=(prob < 0.5),alpha=0.15,color='red')
plt.xlabel('z (linear combination of features)')
plt.ylabel('σ(z) = probability')
plt.title('Sigmoid Function - The Heart of Logistic Regression')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
print(f"sigmoid(0) = {sigmoid(0):.4f}")
print(f"sigmoid(5) = {sigmoid(5):.4f}")
print(f"sigmoid(-5) = {sigmoid(-5):.4f}")
```




```
sigmoid(0) = 0.5000
sigmoid(5) = 0.9933
sigmoid(-5) = 0.0067
```

```
from sklearn import linear_model
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score,recall_score,f1_score
from sklearn.model_selection import train_test_split
```

```
x,y=make_classification(n_samples=500,n_features=5,
                        weights=[0.7,0.3],random_state=42)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
model=LogisticRegression();
model.fit(x_train,y_train)
probs=model.predict_proba(x_test)[:,1]
```

```
print(f'{'threshold':>10}{'precision':>10}{'recall':>8}{'f1':>66}')
print('_'*45)
for threshold in[0.2,0.3,0.4,0.5,0.6,0.7,0.8]:
  preds=(probs>=threshold).astype(int)
  precision=precision_score(y_test,preds,zero_division=0)
  recall=recall_score(y_test,preds)
  f1=f1_score(y_test,preds,zero_division=0)
  print(f'{threshold:>10.1f}{precision:>10.3f}{recall:>8.3f}{f1:>6.3f}')
```

```
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
y_true=[1,1,1,1,1 ,   0,0,0,0,0,  1,0,1,0,1]
y_true=np.array(y_true)
y_pred=[1,1,1,0,0 , 0,0,0,1,1,    1,0,0,1,1 ]
y_pred=np.array(y_pred)
TP=((y_true==1) & (y_pred==1)).sum()
TN=((y_true==0) & (y_pred==0)).sum()
FP=((y_true==0) & (y_pred==1)).sum()
FN=((y_true==1) & (y_pred==0)).sum()
print(f'TP:{TP},TN:{TN},FP:{FP}, FN:{FN}')
print(f'accuracy:{(TP+TN)/total=
 ({TP}+{TN}/{len(y_true)}={(TP+TN)/len(y_true):.2%')
print(f'precision:{TP/(TP+FP)}')
print(f'recall:{TP/(TP+FN)}')
```

```
  File "/tmp/ipykernel_2079/1823018607.py", line 11
    ({TP}+{TN}/{len(y_true)}={(TP+TN)/len(y_true):.2%')
    ^
SyntaxError: f-string: expecting '!', or ':', or '}'
```

```
cn=confusion_matrix(y_true,y_pred)
print('confusion matrix:')
print(cn)
disp=ConfusionMatrixDisplay(confusion_matrix=cn,display_labels=['Negative '])
disp.plot(cnap='blues',colorbar=False)
plt.title('confusion matrix')
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_2079/3272333982.py in <cell line: 0>()
----> 1 cn=confusion_matrix(y_true,y_pred)
      2 print('confusion matrix:')
      3 print(cn)
      4 disp=ConfusionMatrixDisplay(confusion_matrix=cn,display_labels=['Negative '])
      5 disp.plot(cnap='blues',colorbar=False)

NameError: name 'confusion_matrix' is not defined
```

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report,f1_score,accuracy_score
from sklearn.model_selection import train_test_split
import numpy as np
x_bal,y_bal=make_classification(n_samples=1000,weights=[0.5,0.5],random_state=1)
x_imb,y_imb=make_classification(n_samples=1000,weights=[0.99,0.1],random_state=1)
x_bal_train,x_bal_test,y_bal_train,y_bal_test=train_test_split(x_bal,y_bal,test_size=0.2,random_state=1)
```

```
def evaluate(x,y,label):
  x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
  model=LogisticRegression(max_iter=1000)
  model.fit(x_train,y_train)
  y_pred=model.predict(x_test)
  acc=accuracy_score(y_test,y_pred)
  f1=f1_score(y_test,y_pred,zero_division=0)
  print(f'\n==={label}===')
  print(f'accuracy:{acc:.2%}{'misleading'if f1<0.5 else 'ok'}')
  print(f'f1 score:{f1:.2%} real performance')
  print(classification_report(y_test,y_pred,zero_division=0))
```

```
x_train,x_test,y_train,y_test=train_test_split(x_imb,y_imb,test_size=0.2,random_state=42)
model=LogisticRegression(class_weight='balanced',max_iter=1000)
```

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_curve,roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
x,y=make_classification(n_samples=1000,n_features=10,random_state=42)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
scaler=StandardScaler()
x_train=scaler.fit_transform(x_train)
x_test=scaler.transform(x_test)
```

```
lr=LogisticRegression().fit(x_train,y_train)
prob_lr=lr.predict_proba(x_test)[:,1]
dt=DecisionTreeClassifier().fit(x_train,y_train)
prob_dt=dt.predict_proba(x_test)[:,1]
FPR_lr,TPR_lr,_=roc_curve(y_test,prob_lr)
FPR_dt,TPR_dt,_=roc_curve(y_test,prob_dt)
x,y=make_classification(n_samples=800,n_features=10,random_state=42)
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
scaler=StandardScaler()
x_train=scaler.fit_transform(x_train)
x_test=scaler.transform(x_test)
lr=LogisticRegression().fit(x_train,y_train)
prob=lr.predict_proba(x_test)[:,1]
dt=DecisionTreeClassifier().fit(x_train,y_train)
prob_dt=dt.predict_proba(x_test)[:,1]
fpr_lr, tpr_lr, _ = roc_curve(y_test, prob_lr)
fpr_dt, tpr_dt, _ = roc_curve(y_test, prob_dt)
```

```
auc_lr = roc_auc_score(y_test, prob_lr)
auc_dt = roc_auc_score(y_test, prob_dt)
plt.figure(figsize=(7, 5))
plt.plot(fpr_lr, tpr_lr, lw=2.5, color='steelblue', label=f'Logistic Reg AUC={auc_lr:.3f}')
plt.plot(fpr_dt, tpr_dt, lw=2.5, color='tomato', label=f'Decision Tree AUC={auc_dt:.3f}')
plt.plot([0,1],[0,1], 'k--', alpha=0.5, label='Random Classifier AUC=0.50')
plt.fill_between(fpr_lr, tpr_lr, alpha=0.1, color='steelblue')
plt.xlabel('False Positive Rate (FPR)', fontsize=11)
plt.ylabel('True Positive Rate / Recall (TPR)', fontsize=11)
plt.title('ROC Curve — Model Comparison', fontsize=13)
plt.legend(fontsize=10); plt.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```